Dokumentation

Team: Gruppe C
Name: Dennis Sergeev
23.03.2026

Inhaltsverzeichnis

Lösungsidee....1
Umsetzung....1
Beispiele....2
Quellcode....2

Lösungsidee

Idee: In einem Objekt soll eine ArrayList gespeichert werden, die den Bestand des Produkts beinhaltet, (Produkt-ID/Produktname/Produktpreis/Produktbestand).

In einem zweiten Objekt soll eine ArrayList gespeichert werden, die die Funktionen eines Warenkorbs hat. Dort findet man alle Bestellungen (gebuchte/ungebuchte). Der Warenkorb beinhaltet, (Bestellungs-Nr. / Bestellung wurde gebucht / Produkt-ID / Produktname / Produktpreis / Produktbestand).

Änderungen werden in der Bestandsliste aktualisiert (Objekt 1).

Buchungen werden in den Warenkorb hinzugefügt/gelöscht (Objekt 2).

Umsetzung

In ein Objekt soll eine ArrayList gespeichert werden, die den Bestand des Produkts beinhaltet (Produkt-ID = int / Produkt-Name = String / Produkt-Preis = float / Produkt-Bestand = int).

In einem zweiten Objekt soll eine ArrayList gespeichert werden, die die Funktionen eines Warenkorbs hat. Dort findet man alle Bestellungen (gebuchte/ungebuchte). Der Warenkorb beinhaltet, (Bestellungs-Nr. = int / Bestellung wurde gebucht = boolean / Produkt-ID = int / Produkt-Name = String / Produkt-Preis = float / Produkt-Bestand = int).

Änderungen werden in der Bestandsliste aktualisiert (Objekt 1).

Buchungen werden in den Warenkorb hinzugefügt/gelöscht (Objekt 2).

Beim Löschen wird die aktuelle Bestellungs-Nr. gesucht. Dabei wird überprüft, ob die Bestellung gebucht wurde. Wenn die gebuchte Bestellung = false ist, kann man die Bestellung löschen. Gebuchte Bestellungen, die true sind, kann man nicht mehr löschen.

Bei „Bestellung anzeigen“ wird eine for-Schleife aufgerufen, die alle Bestellungen in Objekt2 anzeigt.

Bestellung buchen setzt die Variable „Bestellung wurde gebucht“ auf true.

Programm beenden, setzt die While-Schleife auf false.

Werkzeuge

Website, W3Schools: Java-Logik wie try-catch (Exceptions) / ArrayList / switch-case

Wissen wurde in den Daten ProductList.java und Menu.java verwendet

Buch, Java von Kopf bis Fuß, 3. Auflage: Getter/Setter/Constructor

Wissen wurde in den Dateien OrderNummber.java und ProductList.java und Product.java verwendet

Gemini 3 Pro: Erstellung von Code und Überarbeitung von Logikfehlern

Überarbeitung von Logikfehlern mit Gemini.

```java
            // Innerhalb von Case 2 -> Case 1 (Menge ändern):
            case 1:
                System.out.print("Produkt ID zum Ändern: ");
                int editID = scr.nextInt();
                boolean gefunden = false;
                for (OrderNummber on : OrderNummberList) {
                    if (on.getOrderNumm() == aktuelleBestellNummer && on.getProductId() == editID && !on.getOrderCompleted()) {
                        System.out.print("Neue Menge: ");
                        int newQty = scr.nextInt();
                        int diff = newQty - on.getQuantity();

                        if (list.getStock(editID) >= diff) {
                            list.setStock(editID, newStock: list.getStock(editID) - diff);
                            on.setQuantity(newQty);
                            System.out.println("Menge angepasst.");
                        } else {
                            System.out.println("Lagerbestand reicht nicht!");
                        }
                        gefunden = true;
                        break;
                    }
                }
                if (!gefunden) System.out.println("Position nicht in dieser Bestellung gefunden.");
                break;

            case 2: // Position löschen
                System.out.print("Produkt ID zum Entfernen aus dieser Bestellung: ");
                int remID = scr.nextInt();
                for (int i = OrderNummberList.size() - 1; i >= 0; i--) {
                    OrderNummber on = OrderNummberList.get(i);
                    if (on.getOrderNumm() == aktuelleBestellNummer && on.getProductId() == remID) {
                        // Bestand zurückgeben
                        list.setStock(remID, newStock: list.getStock(remID) + on.getQuantity());
                        OrderNummberList.remove(i);
                        System.out.println("Position entfernt.");
                        break;
                    }
                }
                break;

            case 3: // Raus aus Unter Menu
                subMenuRunning = false;
                break;

            default:
                System.out.println("Ungültige Wahl.");
        }
    }
break;
```

Überarbeitung von Logikfehlern mit Gemini.

```java
case 3:

    if (!OrderNummberList.isEmpty()) {
        //Die Nummer der letzten Bestellung herausfinden
        int gesuchteNummer = OrderNummberList.get(OrderNummberList.size() - 1).getOrderNumm();
        boolean wurdeGeloescht = false;

        //Die Liste RÜCKWÄRTS durchgehen
        for (int i = OrderNummberList.size() - 1; i >= 0; i--) {
            OrderNummber aktuelle = OrderNummberList.get(i);

            // Nur löschen, wenn Nummer stimmt und sie noch nicht abgeschlossen ist
            if (aktuelle.getOrderNumm() == gesuchteNummer) {
                if (!aktuelle.getOrderCompleted()) { // Nur wenn false

                    //Bestand Zurückgeben
                    int aktuellerLagerBestand = list.getStock(aktuelle.getProductId());
                    int neueMenge = aktuellerLagerBestand + aktuelle.getQuantity();
                    list.setStock(aktuelle.getProductId(), neueMenge);
                    // ----------------------------------

                    OrderNummberList.remove(i);
                    wurdeGeloescht = true;
                } else {
                    System.out.println("Bestellung Nr. " + gesuchteNummer + " ist bereits abgeschlossen und kann nicht gelöscht werden!");
                    //Abgeschlossene Bestellung darf man nicht löschen auf der Liste
                    break;
                }
            }
        }

        if (wurdeGeloescht) {
            System.out.println("Bestellung Nr. " + gesuchteNummer + " wurde gelöscht und Bestand korrigiert.");
        }
    } else {
        System.out.println("Keine Bestellungen vorhanden.");
    }

break;
```

Hilfe von Gemini bekommen, bei der Erstellung von Code.

```java
case 5:
    if (!OrderNummberList.isEmpty()) {
        //Nummer der letzten Bestellung herausfinden
        int gesuchteNummer = OrderNummberList.get(OrderNummberList.size() - 1).getOrderNumm();

        // Prüfen, ob die Bestellung bereits bezahlt wurde
        if (OrderNummberList.get(OrderNummberList.size() - 1).getOrderCompleted()) {
            System.out.println("Diese Bestellung (Nr. " + gesuchteNummer + ") wurde bereits bezahlt.");
            break;
        }

        // Gesamtpreis berechnen
        float gesamtPreis = 0;
        for (OrderNummber on : OrderNummberList) {
            if (on.getOrderNumm() == gesuchteNummer) {
                // Preis aus der ProductList holen (Preis * Menge)
                float einzelPreis = list.inventory.get(on.getProductId()).getPrice();
                gesamtPreis += (einzelPreis * on.getQuantity());
            }
        }

        System.out.printf("Gesamtbetrag für Bestellung Nr. % d: %.2f€\n", gesuchteNummer, gesamtPreis);
        System.out.print("Gegebenes Geld vom Kunden: ");
        float gegebenesGeld = scr.nextFloat();

        //Zahlung prüfen
        if (gegebenesGeld < gesamtPreis) {
            //Wenn, zu wenig Geld
            float fehlt = gesamtPreis - gegebenesGeld;
            System.out.printf("Das reicht nicht! Es fehlen noch %.2f€\n", fehlt);
        } else {
            //Wenn, genug oder mehr Geld (Rückgeld)
            if (gegebenesGeld > gesamtPreis) {
                float rueckgeld = gegebenesGeld - gesamtPreis;
                System.out.printf("Zahlung erfolgreich. Rückgeld: %.2f€\n", rueckgeld);
            } else {
                System.out.println("Zahlung erfolgreich. Passend bezahlt.");
            }

            //Alle Positionen dieser Bestellung auf abgeschlossen setzen
            for (OrderNummber on : OrderNummberList) {
                if (on.getOrderNumm() == gesuchteNummer) {
                    on.setOrderCompleted(true);
                }
            }
            System.out.println("Bestellung Nr. " + gesuchteNummer + " wurde gebucht.");
            orderNumm++;
            // In Case 5, nachdem die Zahlung erfolgreich war:
            tagesUmsatz += gesamtPreis;
        }
    } else {
        System.out.println("Keine Bestellungen vorhanden.");
    }

break;
```

Hilfe von Gemini bekommen, bei der Erstellung von Code.

```java
case 6:
    boolean adminMenuRunning = true;
    while (adminMenuRunning) {
        System.out.println("\n--- ADMIN BEREICH (Tagesüberblick) ---");
        printAdminMenu();
        System.out.print("Wahl: ");
        int adminChoice = scr.nextInt();

        if (adminChoice == 0) {
            System.out.println("=====================================");
            System.out.println("      Tagesstatistik Event-Kasse      ");
            System.out.println("=====================================");

            // Umsatz anzeigen
            System.out.printf("Gesamtumsatz heute: %.2f€\n", tagesUmsatz);

            System.out.println("Abgeschlossene Bestellungen: " + orderNumm);

            //Kritische Bestände anzeigen (Anforderung "knappste Artikel")
            System.out.println("\nKritische Bestände (weniger als 3 Stück):");
            boolean allesVoll = true;
            for (int i = 0; i < list.inventory.size(); i++) {
                if (list.getStock(i) < 3) {
                    System.out.println("Warnung " + list.inventory.get(i).getName() + ": nur noch " + list.getStock(i) + " Übrig!");
                    allesVoll = false;
                }
            }
            if (allesVoll) {
                System.out.println("Alle Artikel sind ausreichend vorhanden.");
            }
            System.out.println("=====================================");

        } else if (adminChoice == 1) {
            adminMenuRunning = false; // Zurück zum Hauptmenü
        } else {
            System.out.println("Ungültige Option.");
        }
    }
    break;
```

Beispiele

Um die Produktliste anzuzeigen, 0 in der Konsole eingeben.

Mit 1 eine Bestellung anlegen (Produkt-ID eingeben (findet man in der Produktliste) und die Menge)

Wenn man mehrere Produkte in einer Bestellung haben will, drückt man auf die 2 „Bestellung bearbeiten“. Hier kann man mit 0, Positionen hinzufügen, 1 Positionen bearbeiten, 2 Positionen löschen und 3 wieder ins Main-Menu gelangen.

Falls man die ganze Bestellung löschen will, dann in die Konsole 3 eingeben.

Mit 4 werden alle Bestellungen angezeigt (es werden keine gelöschten Bestellungen angezeigt).

Mit 5 wird die Bestellung gebucht.

Hier nochmals eine bildliche Veranschaulichung:

```
Wähle ein Menupunkt aus: 0

ID: 0 | Kebab | 3.2€ | Bestand: 30
ID: 1 | Pizza | 3.99€ | Bestand: 12
ID: 2 | Hamburger | 0.99€ | Bestand: 8
ID: 3 | Fries | 1.49€ | Bestand: 22
ID: 4 | Chicken Nuggets Bucket | 8.3€ | Bestand: 6
ID: 5 | Coocked Potato | 2.0€ | Bestand: 123
ID: 6 | Baklava | 2.89€ | Bestand: 16
ID: 7 | Lahmacun | 1.67€ | Bestand: 14

Wähle ein Menupunkt aus: 1
Produkt ID: 0
Menge: 2
Bestellung Nr. 0 angelegt.

Wähle ein Menupunkt aus: 2

--- Bearbeite Bestellung Nr. 0 ---
0 | Position hinzufügen
1 | Position bearbeiten
2 | Position löschen
3 | Raus aus Unter Menu
Wähle eine Option: 0
Produkt ID zum Hinzufügen: 2
Menge: 3
Hamburger hinzugefügt.

--- Bearbeite Bestellung Nr. 0 ---
0 | Position hinzufügen
1 | Position bearbeiten
2 | Position löschen
3 | Raus aus Unter Menu
Wähle eine Option: 3

Wähle ein Menupunkt aus: 4
----Deine Bestellungen----
Bestellungsnummer: 0 Bestellung abgeschlossen: false Produkt: Kebab Mänge: 2
Bestellungsnummer: 0 Bestellung abgeschlossen: false Produkt: Hamburger Mänge: 3
```

```
Wähle ein Menupunkt aus: 2

--- Bearbeite Bestellung Nr. 0 ---
0 | Position hinzufügen
1 | Position bearbeiten
2 | Position löschen
3 | Raus aus Unter Menu
Wähle eine Option: 1
Produkt ID zum Ändern: 2
Neue Menge: 5
Menge angepasst.

--- Bearbeite Bestellung Nr. 0 ---
0 | Position hinzufügen
1 | Position bearbeiten
2 | Position löschen
3 | Raus aus Unter Menu
Wähle eine Option: 2
Produkt ID zum Entfernen aus dieser Bestellung: 0
Position entfernt.

--- Bearbeite Bestellung Nr. 0 ---
0 | Position hinzufügen
1 | Position bearbeiten
2 | Position löschen
3 | Raus aus Unter Menu
Wähle eine Option: 3

Wähle ein Menupunkt aus: 4
----Deine Bestellungen----
Bestellungsnummer: 0 Bestellung abgeschlossen: false Produkt: Hamburger Mänge: 5

Wähle ein Menupunkt aus:
```

```
Wähle ein Menupunkt aus: 5
Gesamtbetrag für Bestellung Nr.  0: 4.95€
Gegebenes Geld vom Kunden: 15.36
Zahlung erfolgreich. Rückgeld: 10.41€
Bestellung Nr. 0 wurde gebucht.

Wähle ein Menupunkt aus: 4
----Deine Bestellungen----
Bestellungsnummer: 0 Bestellung abgeschlossen: true Produkt: Hamburger Mänge: 5

Wähle ein Menupunkt aus:
```

```
Wähle ein Menupunkt aus: 6

--- ADMIN BEREICH (Tagesüberblick) ---
0 | Tagesüberblick
1 | Raus aus Unter Menu
Wahl: 0
=====================================
      Tagesstatistik Event-Kasse
=====================================
Gesamtumsatz heute: 4.95€
Abgeschlossene Bestellungen: 2

Kritische Bestände (weniger als 3 Stück):
Alle Artikel sind ausreichend vorhanden.
=====================================

--- ADMIN BEREICH (Tagesüberblick) ---
0 | Tagesüberblick
1 | Raus aus Unter Menu
Wahl: 1

Wähle ein Menupunkt aus: 7

Process finished with exit code 0
```

Fall Bestellung löschen:

```
0 | Artikel liste Anzeigen
1 | Bestellung anlegen
2 | Bestellung bearbeiten
3 | Bestellung löschen
4 | Bestellung anzeigen
5 | Bestellung buchen
6 | Admin Menu
7 | Programm beenden

Wähle ein Menupunkt aus: 0

ID: 0 | Kebab | 3.2€ | Bestand: 30
ID: 1 | Pizza | 3.99€ | Bestand: 12
ID: 2 | Hamburger | 0.99€ | Bestand: 8
ID: 3 | Fries | 1.49€ | Bestand: 22
ID: 4 | Chicken Nuggets Bucket | 8.3€ | Bestand: 6
ID: 5 | Coocked Potato | 2.0€ | Bestand: 123
ID: 6 | Baklava | 2.89€ | Bestand: 16
ID: 7 | Lahmacun | 1.67€ | Bestand: 14

Wähle ein Menupunkt aus: 1
Produkt ID: 0
Menge: 23
Bestellung Nr. 0 angelegt.

Wähle ein Menupunkt aus: 0

ID: 0 | Kebab | 3.2€ | Bestand: 7
ID: 1 | Pizza | 3.99€ | Bestand: 12
ID: 2 | Hamburger | 0.99€ | Bestand: 8
ID: 3 | Fries | 1.49€ | Bestand: 22
ID: 4 | Chicken Nuggets Bucket | 8.3€ | Bestand: 6
ID: 5 | Coocked Potato | 2.0€ | Bestand: 123
ID: 6 | Baklava | 2.89€ | Bestand: 16
ID: 7 | Lahmacun | 1.67€ | Bestand: 14

Wähle ein Menupunkt aus: 3
Bestellung Nr. 0 wurde gelöscht und Bestand korrigiert.

Wähle ein Menupunkt aus: 0

ID: 0 | Kebab | 3.2€ | Bestand: 30
ID: 1 | Pizza | 3.99€ | Bestand: 12
ID: 2 | Hamburger | 0.99€ | Bestand: 8
ID: 3 | Fries | 1.49€ | Bestand: 22
ID: 4 | Chicken Nuggets Bucket | 8.3€ | Bestand: 6
ID: 5 | Coocked Potato | 2.0€ | Bestand: 123
ID: 6 | Baklava | 2.89€ | Bestand: 16
ID: 7 | Lahmacun | 1.67€ | Bestand: 14

Wähle ein Menupunkt aus: 4
Keine Bestellungen angelegt.

Wähle ein Menupunkt aus: |
```

Buchung mehrerer Artikel:

```
Wähle ein Menupunkt aus: 4
----Deine Bestellungen----
Bestellungsnummer: 0 Bestellung abgeschlossen: false Produkt: Kebab Mänge: 5
Bestellungsnummer: 0 Bestellung abgeschlossen: false Produkt: Pizza Mänge: 3
Bestellungsnummer: 0 Bestellung abgeschlossen: false Produkt: Baklava Mänge: 2

Wähle ein Menupunkt aus: 5
Gesamtbetrag für Bestellung Nr.  0: 33.75€
Gegebenes Geld vom Kunden: 100.77
Zahlung erfolgreich. Rückgeld: 67.02€
Bestellung Nr. 0 wurde gebucht.

Wähle ein Menupunkt aus: 4
----Deine Bestellungen----
Bestellungsnummer: 0 Bestellung abgeschlossen: true Produkt: Kebab Mänge: 5
Bestellungsnummer: 0 Bestellung abgeschlossen: true Produkt: Pizza Mänge: 3
Bestellungsnummer: 0 Bestellung abgeschlossen: true Produkt: Baklava Mänge: 2

Wähle ein Menupunkt aus: 1
Produkt ID: 3
Menge: 3
Bestellung Nr. 2 angelegt.

Wähle ein Menupunkt aus: 4
----Deine Bestellungen----
Bestellungsnummer: 0 Bestellung abgeschlossen: true Produkt: Kebab Mänge: 5
Bestellungsnummer: 0 Bestellung abgeschlossen: true Produkt: Pizza Mänge: 3
Bestellungsnummer: 0 Bestellung abgeschlossen: true Produkt: Baklava Mänge: 2
Bestellungsnummer: 2 Bestellung abgeschlossen: false Produkt: Fries Mänge: 3

Wähle ein Menupunkt aus: 7
```